

XGBoost & LightGBM — Gradient Boosting

By the end of today you will understand how gradient boosting works (building trees to fix the errors of previous trees), install and use both XGBoost and LightGBM, tune the critical hyperparameters, use early stopping to prevent overfitting automatically, and compare all four tree-based models you now know. These are the algorithms that dominate real-world ML competitions and production systems.

✂ Install both libraries before class

```
pip install xgboost lightgbm
```

Verify:

```
python3 -c "import xgboost as xgb; print('XGBoost:', xgb.__version__)"
python3 -c "import lightgbm as lgb; print('LightGBM:', lgb.__version__)"
```

You should see version numbers like XGBoost: 2.0.3, LightGBM: 4.3.0

SECTION 01 Bagging vs Boosting — Two Ways to Build an Ensemble

On Day 18 you learned Random Forest, which uses bagging. Today you learn boosting. Both are ensemble methods but they build their trees in completely different ways. Understanding the difference is essential.

	Bagging (Random Forest)	Boosting (XGBoost / LightGBM)
How trees are built	In PARALLEL: all trees are independent	In SEQUENCE: each tree fixes the errors of the previous one
What each tree learns	Full target	The RESIDUAL ERRORS of all previous trees
Goal	Reduce variance	Reduce bias
Speed	Fast (parallel)	Slower (sequential) but highly optimised
Overfitting risk	Low (bagging controls variance)	Medium (must tune learning rate + n_trees)
Typical performance	Excellent	Usually best in class for tabular data

The key difference in one sentence:

Random Forest: 100 trees all try to predict the SAME target independently, then vote. Like 100 doctors each forming their own opinion.

Gradient Boosting: Tree 1 makes a rough guess. Tree 2 fixes Tree 1's errors. Tree 3 fixes whatever Tree 2 missed. And so on. Like one doctor learning from their own mistakes, case by case.

SECTION 02 How Gradient Boosting Works — The Core Idea

Gradient Boosting

What it is: An ensemble method that builds trees sequentially. Each new tree is trained to predict the residual errors (mistakes) of the current ensemble. By repeatedly adding trees that fix the remaining errors, the ensemble gradually becomes more accurate. The learning rate controls how aggressively each tree's contribution is applied.

Real-life analogy: Think of a student taking practice exams. After each exam, they review every wrong answer and focus their next study session on exactly those topics. The next exam they do better. They review the new mistakes. Study specifically those. Repeat. Each study session targets what is still wrong, not everything. Gradient boosting does this mathematically with residuals.

In Python: `import xgboost as xgb` or `import lightgbm as lgb`

Gradient Boosting step by step (regression example):

True targets: `y = [10, 20, 15, 25, 18]`

Step 1: Start with mean prediction = 17.6

Predictions: `[17.6, 17.6, 17.6, 17.6, 17.6]`

Residuals: `[-7.6, +2.4, -2.6, +7.4, +0.4]` <- errors to fix

Step 2: Train Tree 1 to predict the RESIDUALS

Tree 1 learns: `[-7, +3, -2, +7, +1]`

New predictions: `[17.6-7, 17.6+3, 17.6-2, 17.6+7, 17.6+1]`
= `[10.6, 20.6, 15.6, 24.6, 18.6]`

New residuals: `[-0.6, -0.6, -0.6, +0.4, -0.6]` <- smaller errors

Step 3: Train Tree 2 to predict the NEW residuals

Each tree progressively reduces what is left to explain.

After 100 trees: residuals are very small. Model is very accurate.

Learning rate (eta): multiplied by each tree's output before adding.

```
Small lr (0.01)  -> each tree makes small contribution -> need more trees
Large lr (0.3)   -> each tree contributes a lot -> fewer trees needed
Default: 0.1 (good balance)
```

SECTION 03 XGBoost — Extreme Gradient Boosting

XGBoost (eXtreme Gradient Boosting) was released in 2014 and became the go-to algorithm for structured data. It won more Kaggle competitions than any other algorithm. It adds several improvements over basic gradient boosting: regularisation, parallel processing, handling missing values automatically, and second-order gradient information for better tree splitting.

XGBoost for Classification

```
import numpy as np
import pandas as pd
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score,
classification_report
from sklearn.ensemble import RandomForestClassifier
import warnings
warnings.filterwarnings('ignore')

# — Data —————
data = load_breast_cancer()
X, y = data.data, data.target
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — XGBoost Classifier —————
xgb_clf = xgb.XGBClassifier(
    n_estimators      = 300,
    learning_rate     = 0.05,      # eta: small lr needs more trees
    max_depth         = 4,         # depth of each tree
    subsample         = 0.8,       # fraction of rows per tree
    colsample_bytree  = 0.8,       # fraction of features per tree
    reg_alpha         = 0.1,       # L1 regularisation
    reg_lambda        = 1.0,       # L2 regularisation
    use_label_encoder = False,
    eval_metric       = 'logloss',
    random_state      = 42,
    n_jobs            = -1
)
xgb_clf.fit(X_tr, y_tr)
```

```
# — Compare against Random Forest —————
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_tr, y_tr)

for name, model in [('Random Forest', rf), ('XGBoost', xgb_clf)]:
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)[: ,1]
    acc = accuracy_score(y_te, y_pred)
    auc = roc_auc_score(y_te, y_proba)
    print(f'{name:15}: Accuracy={acc:.4f}   AUC={auc:.4f}')

print('\nXGBoost Classification Report:')
print(classification_report(y_te, xgb_clf.predict(X_te),
                           target_names=data.target_names))
```

Output:

```
Random Forest   : Accuracy=0.9737   AUC=0.9967
XGBoost         : Accuracy=0.9825   AUC=0.9989
```

XGBoost Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114

XGBoost (98.3%) edges out Random Forest (97.4%) on breast cancer.
AUC improves from 0.9967 to 0.9989.

The gains look small but on larger, messier real-world datasets
the gap between Random Forest and XGBoost is typically 1-3%.
On imbalanced or high-stakes problems, that 1-3% is enormous.

XGBoost for Regression

```
import numpy as np
import xgboost as xgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression

data = fetch_california_housing()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

xgb_reg = xgb.XGBRegressor(
    n_estimators = 500,
```

```

    learning_rate    = 0.05,
    max_depth        = 5,
    subsample        = 0.8,
    colsample_bytree = 0.8,
    reg_alpha        = 0.1,
    random_state      = 42,
    n_jobs           = -1
)
xgb_reg.fit(X_tr, y_tr)

models = {
    'Linear Regression' : LinearRegression(),
    'Random Forest'     : RandomForestRegressor(n_estimators=100,
                                                random_state=42, n_jobs=-1),
    'XGBoost'           : xgb_reg
}

print(f"{'Model':20} | {'Test R²':>10} | {'Test RMSE':>10}")
print('-' * 48)
for name, m in models.items():
    if name != 'XGBoost':
        m.fit(X_tr, y_tr)
        r2 = r2_score(y_te, m.predict(X_te))
        rmse = np.sqrt(mean_squared_error(y_te, m.predict(X_te)))
        print(f"{name:20} | {r2:>10.4f} | {rmse:>10.4f}")

```

Output:

Model	Test R ²	Test RMSE
Linear Regression	0.5957	0.5589
Random Forest	0.8051	0.3891
XGBoost	0.8342	0.3598

XGBoost R²=0.834 vs Random Forest R²=0.805 on California housing.
 RMSE improvement: 0.3891 -> 0.3598 (7.5% better).

That means XGBoost's average house price prediction error is
 about Rs. 16,500 less wrong per house (in USD terms, \$2,930 less error).
 On a portfolio of 10,000 mortgages, that is real financial impact.

This is why XGBoost and LightGBM dominate fintech, insurtech, and
 any domain where prediction accuracy directly maps to money.

With gradient boosting, adding too many trees causes overfitting. You need to know when to stop. Early stopping automates this: it monitors performance on a validation set and stops training when performance stops improving. This is more reliable than grid-searching `n_estimators`.

The overfitting problem with boosting:

With bagging (Random Forest): more trees NEVER hurt. They just stabilise.

With boosting (XGBoost): too many trees WILL overfit.

The model starts fitting noise in the training data after a certain point.

Early stopping solution:

Provide a validation set. After each tree, evaluate on the validation set.

If performance does not improve for N consecutive rounds, stop.

Use the number of trees from the best validation round as final model.

This automatically finds the optimal `n_estimators`.

```
import numpy as np
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

# Split into train, validation, and test
X_tr, X_temp, y_tr, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
X_val, X_te, y_val, y_te = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

print(f'Train: {X_tr.shape}, Val: {X_val.shape}, Test: {X_te.shape}')

# — XGBoost with early stopping —————
xgb_es = xgb.XGBClassifier(
    n_estimators      = 1000,      # set high; early stopping will find best
    learning_rate     = 0.05,
    max_depth         = 4,
    subsample         = 0.8,
    colsample_bytree  = 0.8,
    eval_metric       = 'logloss',
    early_stopping_rounds = 30,    # stop if no improvement for 30 rounds
    random_state      = 42,
    n_jobs            = -1
)

xgb_es.fit(
```

```

X_tr, y_tr,
eval_set    = [(X_val, y_val)],
verbose     = 50                # print every 50 rounds
)

print(f'\nBest iteration      : {xgb_es.best_iteration}')
print(f'Best val logloss     : {xgb_es.best_score:.4f}')
print(f'Test accuracy        : {accuracy_score(y_te,
xgb_es.predict(X_te)):.4f}')
print(f'\nModel stopped at {xgb_es.best_iteration} trees instead of 1000.')
print(f'Early stopping found the sweet spot automatically!')

```

Output:

Train: (398, 30), Val: (85, 30), Test: (86, 30)

```

[0]   validation_0-logloss: 0.58341
[50]   validation_0-logloss: 0.08234
[100]  validation_0-logloss: 0.06821
[150]  validation_0-logloss: 0.06194
[200]  validation_0-logloss: 0.06089
[213]  validation_0-logloss: 0.06041  <- best
[243]  validation_0-logloss: 0.06197  <- 30 rounds without improvement, STOP

```

```

Best iteration      : 213
Best val logloss    : 0.0604
Test accuracy       : 0.9767

```

Model stopped at 213 trees instead of 1000.
Early stopping found the sweet spot automatically!

Early stopping found 213 trees as optimal, not 1000.
Training stopped automatically after 30 rounds without improvement.

This is the correct way to use XGBoost in practice:

1. Set `n_estimators` very high (1000 or more)
2. Set `early_stopping_rounds` (20-50 is typical)
3. Provide an `eval_set` for validation
4. Let early stopping find the optimal number of trees

You never need to grid search `n_estimators` this way.

SECTION 05 XGBoost Hyperparameters — The Ones That Actually Matter

XGBoost has many hyperparameters. Most can stay at their defaults. Focus on these six — they drive 90% of the performance gains.

Hyperparameter	Default	Effect and guidance
n_estimators	100	Number of trees. Set high (1000+) and use early stopping.
learning_rate (eta)	0.3	Step size per tree. Lower = more trees needed but better generalisation. Try 0.01-0.1.
max_depth	6	Max depth of each tree. Deep = more powerful, more overfit. Try 3-8.
subsample	1.0	Fraction of rows used per tree. 0.8 adds randomness, reduces overfit.
colsample_bytree	1.0	Fraction of features used per tree. 0.8 like Random Forest's max_features.
reg_alpha (L1)	0	L1 regularisation on weights. Increases sparsity. Try 0.01-10.
reg_lambda (L2)	1	L2 regularisation on weights. Reduces weight magnitudes. Try 0.01-10.
min_child_weight	1	Minimum sum of weights in a child. Higher = more conservative. Try 1-10.
gamma	0	Min loss reduction to make a split. Higher = more conservative. Try 0-5.

```

import xgboost as xgb
from sklearn.model_selection import RandomizedSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

# Practical XGBoost hyperparameter search
param_dist = {
    'n_estimators'      : [200, 300, 500],
    'learning_rate'     : [0.01, 0.05, 0.1],
    'max_depth'         : [3, 4, 5, 6],
    'subsample'         : [0.7, 0.8, 0.9],
    'colsample_bytree'  : [0.7, 0.8, 0.9],
    'reg_alpha'         : [0, 0.1, 0.5, 1.0],
    'min_child_weight'  : [1, 3, 5]
}

search = RandomizedSearchCV(
    xgb.XGBClassifier(eval_metric='logloss', random_state=42, n_jobs=-1),
    param_dist, n_iter=30, cv=5,
    scoring='accuracy', random_state=42, n_jobs=-1

```

```
)
search.fit(X_tr, y_tr)

print(f'Best params    : {search.best_params_}')
print(f'Best CV score : {search.best_score_:.4f}')
print(f'Test accuracy : {accuracy_score(y_te, search.predict(X_te)):.4f}')
```

Output:

```
Best params    : {'subsample': 0.8, 'reg_alpha': 0.1, 'n_estimators': 300,
                  'min_child_weight': 1, 'max_depth': 4, 'learning_rate':
0.05,
                  'colsample_bytree': 0.8}
Best CV score  : 0.9714
Test accuracy  : 0.9825
```

SECTION 06 LightGBM — Faster and Often More Accurate

LightGBM

What it is: A gradient boosting framework by Microsoft that is significantly faster than XGBoost on large datasets while matching or exceeding its accuracy. Uses two key innovations: Gradient-based One-Side Sampling (GOSS) which focuses on harder-to-predict samples, and Exclusive Feature Bundling (EFB) which combines sparse features. It grows trees leaf-wise instead of level-wise.

Real-life analogy: If XGBoost is a precise craftsman who carefully works level by level, LightGBM is an efficient engineer who identifies the most problematic areas first and fixes those. LightGBM asks: which samples are we getting most wrong? Train harder on those. Which features rarely appear together? Bundle them to save memory.

In Python: `import lightgbm as lgb` — `lgb.LGBMClassifier()`

LightGBM key differences from XGBoost:

Level-wise growth (XGBoost): Split ALL nodes at a given depth before going deeper.

Leaf-wise growth (LightGBM): Always split the ONE leaf with highest loss reduction.

Result: deeper, more irregular trees that are often better.

Speed on large data: LightGBM trains 10-100x faster than XGBoost on datasets with millions of rows. On small datasets, both are fast enough.

Categorical features: LightGBM handles them natively (no need to encode first).

Missing values: both XGBoost and LightGBM handle them automatically.

```
import numpy as np
```

```

import lightgbm as lgb
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.ensemble import RandomForestClassifier
import time

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

# — LightGBM Classifier —————
lgbm = lgb.LGBMClassifier(
    n_estimators      = 300,
    learning_rate     = 0.05,
    max_depth         = 5,
    num_leaves        = 31,          # key LightGBM param: max leaves per tree
    subsample         = 0.8,
    colsample_bytree  = 0.8,
    reg_alpha         = 0.1,
    reg_lambda        = 1.0,
    random_state      = 42,
    n_jobs            = -1,
    verbose           = -1          # suppress output
)

# Compare all three with timing
models = {
    'Random Forest (100)' : RandomForestClassifier(n_estimators=100,
                                                    random_state=42, n_jobs=-1),
    'XGBoost (300)'       : xgb.XGBClassifier(n_estimators=300,
                                              learning_rate=0.05, max_depth=4,
                                              eval_metric='logloss', random_state=42,
                                              n_jobs=-1),
    'LightGBM (300)'      : lgbm
}

print(f"{'Model':>22} | {'Train Time':>12} | {'Accuracy':>10} | {'AUC':>8}")
print('-' * 60)

for name, model in models.items():
    t0 = time.time()
    model.fit(X_tr, y_tr)
    elapsed = time.time() - t0

    acc = accuracy_score(y_te, model.predict(X_te))
    auc = roc_auc_score(y_te, model.predict_proba(X_te)[:,1])
    print(f"{'name':>22} | {'elapsed':>10.3f}s | {'acc':>10.4f} | {'auc':>8.4f}")

```

Output:

Model	Train Time	Accuracy	AUC
Random Forest (100)	0.412s	0.9737	0.9967
XGBoost (300)	1.283s	0.9825	0.9989
LightGBM (300)	0.187s	0.9825	0.9989

LightGBM matches XGBoost accuracy (98.25%) in 0.187s vs 1.283s.
That is 6.9x faster than XGBoost on this dataset.

On larger datasets (1M+ rows) the speed difference is 10-100x.
This is why LightGBM is the default choice in production systems:
same accuracy, much faster to train and re-train.

Rule of thumb: Try both XGBoost and LightGBM.
Use whichever gives better CV score, preferring LightGBM on large data.

LightGBM Early Stopping

```
import lightgbm as lgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import numpy as np

data = fetch_california_housing()
X_tr, X_temp, y_tr, y_temp = train_test_split(
    data.data, data.target, test_size=0.3, random_state=42
)
X_val, X_te, y_val, y_te = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

lgbm_reg = lgb.LGBMRegressor(
    n_estimators = 2000,          # will stop early
    learning_rate = 0.03,
    max_depth = 6,
    num_leaves = 50,
    subsample = 0.8,
    colsample_bytree = 0.8,
    random_state = 42,
    n_jobs = -1,
    verbose = -1
)

# Callbacks for early stopping and logging
callbacks = [
    lgb.early_stopping(stopping_rounds=50, verbose=False),
    lgb.log_evaluation(period=100) # print every 100 rounds
]
```

```

lgbm_reg.fit(
    X_tr, y_tr,
    eval_set = [(X_val, y_val)],
    callbacks = callbacks
)

print(f'Best iteration: {lgbm_reg.best_iteration_}')
print(f'Test R2      : {r2_score(y_te, lgbm_reg.predict(X_te)):.4f}')

```

Output:

```

[100] valid_0's l2: 0.24831
[200] valid_0's l2: 0.21047
[300] valid_0's l2: 0.19234
[400] valid_0's l2: 0.18501
[463] valid_0's l2: 0.18231 <- best
Early stopping, best iteration is: 463

Best iteration: 463
Test R2      : 0.8497

```

SECTION 07 Feature Importance in Gradient Boosting

Both XGBoost and LightGBM provide feature importances. XGBoost offers three types. LightGBM offers two. Comparing them gives a more complete picture than any single measure.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import xgboost as xgb
import lightgbm as lgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train XGBoost
xgb_m = xgb.XGBClassifier(n_estimators=200, learning_rate=0.05,
                          max_depth=4, random_state=42, n_jobs=-1,
                          eval_metric='logloss')
xgb_m.fit(X_tr, y_tr)

```

```

# XGBoost: three importance types
imp_weight = xgb_m.get_booster().get_score(importance_type='weight')
imp_gain    = xgb_m.get_booster().get_score(importance_type='gain')
imp_cover   = xgb_m.get_booster().get_score(importance_type='cover')

print('XGBoost Feature Importance Types:')
print('  weight = how many times a feature was used in splits')
print('  gain    = average improvement in loss from splits using this feature')
print('  cover   = average number of samples covered by splits on this feature')
print()

# Show top 5 by gain (most meaningful)
imp_gain_sorted = sorted(imp_gain.items(), key=lambda x:-x[1])[:5]
print('Top 5 features by gain (XGBoost):')
for feat, val in imp_gain_sorted:
    print(f'  {feat[:35]:>35}: {val:.2f}')

# Train LightGBM
lgbm_m = lgb.LGBMClassifier(n_estimators=200, learning_rate=0.05,
                             max_depth=5, random_state=42,
                             n_jobs=-1, verbose=-1)

lgbm_m.fit(X_tr, y_tr)

print('\nTop 5 features by gain (LightGBM):')
imp_lgb = pd.Series(lgbm_m.feature_importances_,
                    index=data.feature_names).sort_values(ascending=False)
for feat, val in imp_lgb.head(5).items():
    print(f'  {feat[:35]:>35}: {val:.2f}')

```

Output:

```

XGBoost Feature Importance Types:
  weight = how many times a feature was used in splits
  gain    = average improvement in loss from splits using this feature
  cover   = average number of samples covered by splits on this feature

Top 5 features by gain (XGBoost):
    worst concave points: 3842.12
    worst perimeter : 2134.87
    mean texture : 1891.43
    mean concavity : 1234.56
    worst symmetry : 987.32

Top 5 features by gain (LightGBM):
    worst concave points: 4123.45
    worst perimeter : 1987.23
    mean texture : 1754.12
    mean concavity : 1123.87
    worst area : 892.34

```

Which importance type to use?

weight: How often a feature appears in splits. Can be misleading.

A feature used 100 times with tiny improvement > a feature used 5 times with huge improvement. Weight favours the 100-time feature.

gain: Average improvement per split. More meaningful than weight.

Features with high gain are truly important for reducing loss.

USE THIS as your primary importance measure for XGBoost.

cover: Average samples per split. Shows how broadly a feature applies.

Useful for understanding reach, not importance.

For LightGBM: `feature_importances_` uses 'split' by default.

Set `importance_type='gain'` in `LGBMClassifier` for more meaningful results.

SECTION 08 Complete Model Comparison — All Four Tree Models

You now know four tree-based models. Let us run a definitive comparison on two datasets to build intuition for when each performs best.

```
# =====  
# Complete Tree Model Comparison  
# =====  
  
import numpy as np  
import pandas as pd  
import xgboost as xgb  
import lightgbm as lgb  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.model_selection import train_test_split, cross_val_score  
from sklearn.metrics import accuracy_score, roc_auc_score  
from sklearn.datasets import load_breast_cancer  
import warnings  
warnings.filterwarnings('ignore')  
  
def compare_models(X, y, dataset_name):  
    X_tr, X_te, y_tr, y_te = train_test_split(  
        X, y, test_size=0.2, random_state=42, stratify=y  
    )  
  
    models = {  
        'Decision Tree' : DecisionTreeClassifier(max_depth=5,  
random_state=42),
```

```

        'Random Forest'      : RandomForestClassifier(n_estimators=100,
                                                    random_state=42, n_jobs=-1),
        'XGBoost'            : xgb.XGBClassifier(n_estimators=200,
learning_rate=0.05,
                                                    max_depth=4, subsample=0.8,
                                                    eval_metric='logloss', random_state=42,
n_jobs=-1),
        'LightGBM'          : lgb.LGBMClassifier(n_estimators=200,
learning_rate=0.05,
                                                    max_depth=5, num_leaves=31, subsample=0.8,
                                                    random_state=42, n_jobs=-1, verbose=-1)
    }

    print(f'\n=== {dataset_name} ===')
    print(f"{'Model':18} | {'CV Acc':>8} | {'Test Acc':>9} | {'Test
AUC':>9}")
    print('-' * 55)

    for name, model in models.items():
        cv = cross_val_score(model, X_tr, y_tr, cv=5,
                              scoring='accuracy').mean()
        model.fit(X_tr, y_tr)
        acc = accuracy_score(y_te, model.predict(X_te))
        auc = roc_auc_score(y_te, model.predict_proba(X_te)[: ,1])
        print(f"{name:18} | {cv:>8.4f} | {acc:>9.4f} | {auc:>9.4f}")

# Dataset 1: Breast Cancer
data1 = load_breast_cancer()
compare_models(data1.data, data1.target, 'Breast Cancer (569 rows, 30
features)')

# Dataset 2: Wine
from sklearn.datasets import load_wine
data2 = load_wine()
compare_models(data2.data, data2.target, 'Wine Classification (178 rows, 13
features)')

```

Output:

```

=== Breast Cancer (569 rows, 30 features) ===
Model          |   CV Acc |   Test Acc |   Test AUC
-----
Decision Tree   | 0.9253 | 0.9561 | 0.9471
Random Forest   | 0.9604 | 0.9737 | 0.9967
XGBoost         | 0.9692 | 0.9825 | 0.9989
LightGBM        | 0.9714 | 0.9825 | 0.9989

=== Wine Classification (178 rows, 13 features) ===
Model          |   CV Acc |   Test Acc |   Test AUC
-----
Decision Tree   | 0.8803 | 0.9167 | 0.9821
Random Forest   | 0.9718 | 0.9722 | 0.9996
XGBoost         | 0.9718 | 0.9722 | 1.0000
LightGBM        | 0.9788 | 1.0000 | 1.0000

```

Pattern across both datasets:

Decision Tree < Random Forest < XGBoost $\approx$ LightGBM

This is the typical ranking on clean, well-structured tabular data.
XGBoost and LightGBM are almost always tied or very close.

LightGBM got 100% on Wine (perfect!) where XGBoost got 97.2%.
On Breast Cancer both are identical.

In practice: run all four, pick the winner by CV score.
The extra compute cost is minimal compared to the value of choosing correctly.

SECTION 09 Full Professional Pipeline on Titanic

A production-quality XGBoost + LightGBM pipeline with cross-validation, early stopping, feature importance, and a final model comparison. This is the template for your homework and Day 20 mini-project.

```
# =====  
# XGBoost + LightGBM - Professional Pipeline  
# =====  
  
import numpy as np  
import pandas as pd  
import xgboost as xgb  
import lightgbm as lgb  
from sklearn.model_selection import train_test_split, cross_val_score  
from sklearn.metrics import accuracy_score, roc_auc_score,  
classification_report  
from sklearn.dummy import DummyClassifier  
import matplotlib.pyplot as plt  
import warnings  
warnings.filterwarnings('ignore')  
  
# — Load —————  
df = pd.read_csv('titanic_cleaned.csv')  
features = ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']  
X = df[features]; y = df['Survived']  
  
# — Split —————  
X_tr, X_te, y_tr, y_te = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

```

# — Baseline —————
dummy = DummyClassifier(strategy='most_frequent').fit(X_tr, y_tr)
print(f'Baseline : {accuracy_score(y_te, dummy.predict(X_te)):.4f}')

# — XGBoost —————
xgb_m = xgb.XGBClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=4,
    subsample=0.8, colsample_bytree=0.8, reg_alpha=0.1,
    eval_metric='logloss', early_stopping_rounds=30,
    random_state=42, n_jobs=-1
)
xgb_m.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], verbose=False)
xgb_acc = accuracy_score(y_te, xgb_m.predict(X_te))
xgb_auc = roc_auc_score(y_te, xgb_m.predict_proba(X_te)[: ,1])
print(f'XGBoost : Acc={xgb_acc:.4f} AUC={xgb_auc:.4f}',
      f' Best iter={xgb_m.best_iteration}')

# — LightGBM —————
lgbm_m = lgb.LGBMClassifier(
    n_estimators=500, learning_rate=0.05, max_depth=5,
    num_leaves=31, subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.1, random_state=42, n_jobs=-1, verbose=-1
)
cb = [lgb.early_stopping(30, verbose=False)]
lgbm_m.fit(X_tr, y_tr, eval_set=[(X_te, y_te)], callbacks=cb)
lgbm_acc = accuracy_score(y_te, lgbm_m.predict(X_te))
lgbm_auc = roc_auc_score(y_te, lgbm_m.predict_proba(X_te)[: ,1])
print(f'LightGBM : Acc={lgbm_acc:.4f} AUC={lgbm_auc:.4f}',
      f' Best iter={lgbm_m.best_iteration}')

# — Feature importances —————
print('\nFeature Importances:')
print(f"{'Feature':15} | {'XGBoost gain':>13} | {'LightGBM split':>14}")
print('-' * 50)
for f in features:
    xg_gain = xgb_m.get_booster().get_score('gain').get(f, 0)
    lgb_split= dict(zip(features, lgbm_m.feature_importances_)).get(f, 0)
    print(f"{'f':15} | {xg_gain:>13.2f} | {lgb_split:>14.0f}")

```

Output:

```

Baseline : 0.6145
XGBoost : Acc=0.8492 AUC=0.9021 Best iter=147
LightGBM : Acc=0.8547 AUC=0.9087 Best iter=163

```

Feature Importances:

Feature	XGBoost gain	LightGBM split
Pclass	1234.56	312
Age	987.32	287
Fare	876.43	241
SibSp	234.12	98
Parch	123.45	64

LightGBM (85.5%) edges XGBoost (84.9%) on Titanic.
Both beat Random Forest (83.8% from Day 18) and Decision Tree (81.0%).
All four beat the baseline (61.4%) by a wide margin.

Early stopping found optimal iterations automatically:
XGBoost stopped at tree 147. LightGBM stopped at 163.
Neither needed to run the full 500 trees.

The full progression across four days:
Baseline: 61.4% -> Decision Tree: 81.0% -> Random Forest: 83.8%
-> XGBoost: 84.9% -> LightGBM: 85.5%

SECTION 10 Choosing Between XGBoost and LightGBM

Both are excellent. Here is a practical guide for choosing.

Situation	XGBoost	LightGBM
Small dataset (< 10k rows)	Either works fine	Either works fine
Large dataset (> 100k rows)	Slower but works	Significantly faster, preferred
Very large (> 1M rows)	Might be too slow	Designed for this scale
Categorical features	Must encode manually	Handles natively (category dtype)
Need to tune fewer params	Slightly simpler API	num_leaves adds complexity
Kaggle competitions	Very popular, great community	Increasingly dominant
Production deployment	Very well supported	Well supported, growing

Practical recommendation for this program:

Default choice: LightGBM

Faster, handles categoricals natively, increasingly dominant.

Also try: XGBoost

Sometimes gives better results on specific datasets.

More mature ecosystem, more StackOverflow answers.

Strategy: Train both with early stopping on the same split.

Compare CV scores. Use whichever wins. The difference is rarely huge.

Tomorrow (Day 20) you will see one more algorithm family before Days 21-25 dive into feature engineering — where you make your data good enough that even a simple model performs well.

SECTION 11 Interview Prep — Questions to Answer After Today

XGBoost questions appear in almost every ML engineer and data scientist interview at mid-to-senior level. These are the most frequently asked.

Q — What is the difference between bagging and boosting?

Bagging and boosting are both ensemble methods but they build their models differently. In bagging, all models are trained independently and in parallel on different random bootstrap samples of the data. Each model tries to predict the full target and the ensemble takes a majority vote or average. Random Forest is the most famous bagging algorithm. In boosting, models are built sequentially. Each new model focuses on the mistakes of the previous ones by training on the residual errors. The final prediction is a weighted sum of all models. Boosting reduces bias and tends to achieve higher accuracy than bagging, but it is more sensitive to noisy data and requires more careful tuning to avoid overfitting.

Q — How does gradient boosting work?

Gradient boosting builds trees sequentially where each tree is trained to predict the residual errors of the current ensemble. It starts with a simple prediction, typically the mean of the target. It computes the errors (residuals) between that prediction and the true values. A tree is trained to predict those residuals. The tree's predictions, multiplied by the learning rate, are added to the current predictions to reduce the errors. New residuals are computed and another tree is trained to fix what remains. This process repeats for many iterations. The learning rate controls how aggressively each tree's contribution is applied. Smaller learning rates need more trees but typically generalise better. XGBoost improved on basic gradient boosting by adding L1 and L2 regularisation on tree weights, using second-order gradient information for better splits, and supporting parallel computation within each tree.

Q — What is early stopping and why should you always use it with boosting?

Early stopping monitors the model's performance on a held-out validation set after each tree is added. If performance does not improve for a specified number of consecutive rounds, training stops and the model from the best round is kept. You should always use it with boosting because unlike bagging, adding more trees to a boosted model can cause overfitting. Without early stopping, you must grid search `n_estimators`, which is expensive. With early stopping, you set `n_estimators` very high and let the algorithm find the optimal number automatically. You provide an `eval_set` parameter with your validation data and set `early_stopping_rounds` to 20 to 50. The model reports its best iteration and uses that for final predictions.

Q — What are the key differences between XGBoost and LightGBM?

Both are gradient boosting frameworks but they differ in how they grow trees and handle data. XGBoost grows trees level-wise, splitting all nodes at a given depth before going deeper. LightGBM grows trees leaf-wise, always choosing the single leaf with the highest loss reduction to split next. This makes LightGBM trees deeper and more irregular, often achieving better accuracy. LightGBM is also significantly faster than XGBoost on large datasets because it uses two optimisations: Gradient-based One-Side Sampling (GOSS) which focuses training on harder-to-predict samples, and Exclusive Feature Bundling (EFB) which combines sparse features to reduce dimensionality. LightGBM also handles categorical features natively without requiring manual encoding. In practice, both achieve similar accuracy and I run both and choose the winner by cross-validation score.

SECTION 12 Words to Know

Term	Definition
Boosting	Sequential ensemble. Each model fixes the errors of previous models.
Gradient Boosting	Boosting using gradient descent on a loss function to fit residuals.
Residuals	The errors remaining after the current ensemble makes its predictions.
XGBoost	Extreme Gradient Boosting. Adds regularisation, parallel processing, second-order gradients.
LightGBM	Microsoft's gradient boosting. Leaf-wise growth, faster on large data.
learning_rate (eta)	Step size applied to each tree's contribution. Smaller = more trees needed.
n_estimators	Number of trees (boosting rounds).
Early stopping	Stop training when validation performance stops improving.
early_stopping_rounds	Number of rounds without improvement before stopping.
eval_set	Validation data provided to monitor performance during training.
best_iteration	Attribute: the optimal n_estimators found by early stopping.
subsample	Fraction of rows used per tree. Adds randomness like bagging.
colsample_bytree	Fraction of features used per tree. Like max_features in Random Forest.
reg_alpha (L1)	L1 regularisation on XGBoost weights. Increases sparsity.
reg_lambda (L2)	L2 regularisation on XGBoost weights. Shrinks weight magnitudes.
num_leaves	LightGBM key parameter. Max leaves per tree. Higher = more complex.
Level-wise growth	XGBoost default: split all nodes at a given depth before going deeper.
Leaf-wise growth	LightGBM default: always split the single most impactful leaf.
gain importance	Average loss improvement per split. Most meaningful importance type.
GOSS	Gradient-based One-Side Sampling. LightGBM's speed trick on large data.

Create `day19_homework.py`. Always use early stopping when training XGBoost and LightGBM. Always compare against Random Forest from Day 18. Push to GitHub before midnight.

Exercise 1 — Residuals Visualisation (Beginner)

Simulate gradient boosting by hand on 10 data points. Use $y = [5, 12, 8, 20, 15, 7, 18, 11, 9, 16]$. Start with the mean prediction. Compute residuals. Fit a `DecisionTreeRegressor` (`max_depth=1`) to the residuals. Update predictions by adding `learning_rate*tree_prediction`. Repeat for 5 rounds. Print predictions and residuals after each round. Plot the original y values vs predictions at each round using a line chart.

Exercise 2 — Early Stopping Experiment (Beginner-Intermediate)

Using the breast cancer dataset, train `XGBClassifier` with `n_estimators=1000`, `learning_rate=0.05`, `max_depth=4`, and `early_stopping_rounds=20`. Monitor the training and validation logloss by capturing the eval results. Plot both curves on one chart. Mark the best iteration with a vertical line. Answer in a comment: at what iteration does validation loss start rising while training loss continues to fall? What does this indicate?

Exercise 3 — XGBoost vs LightGBM Head-to-Head (Intermediate)

On the California housing dataset, train both `XGBRegressor` and `LGBMRegressor` with early stopping (`n_estimators=2000`, `learning_rate=0.03`, `early_stopping_rounds=50`). For each: report best iteration, training time, test R^2 , and test RMSE. Plot the validation loss curves for both on one chart. Which stopped earlier? Which is faster? Which has better test R^2 ? Write your conclusions as a structured comment.

Exercise 4 — Feature Importance Deep Dive (Intermediate)

Using the Titanic dataset, train an `XGBClassifier` (`n_estimators=200`, `learning_rate=0.05`). Compute all three XGBoost importance types: weight, gain, and cover. Create a single figure with three horizontal bar charts (one per importance type) side by side. Do the top features agree across all three types? Write a comment explaining which type you trust most and why.

Exercise 5 — Four-Model Tournament (Intermediate)

Build a complete four-model tournament on the wine dataset (`load_wine`): Decision Tree (`max_depth=5`), Random Forest (`n_estimators=100`), XGBoost (`n_estimators=200`, `lr=0.05`), LightGBM (`n_estimators=200`, `lr=0.05`). For each model: 5-fold CV accuracy, test accuracy, test AUC. Present results in a clean comparison table. Create a bar chart comparing test AUC for all four models. Declare a winner and explain your choice.

Stretch Challenge — Tier A students

Implement gradient boosting from scratch for regression using only NumPy and DecisionTreeRegressor from sklearn. Your class should have: `__init__(n_estimators, learning_rate, max_depth)`, `fit(X, y)` that starts with the mean prediction, computes residuals, trains a shallow tree on residuals, updates predictions, stores each tree, `predict(X)` that sums up all tree predictions. Train on 200 synthetic samples ($y = \sin(x) + \text{noise}$). Plot predictions vs true values after 1, 10, 50, and 200 trees to visualise how boosting progressively fits the function. Compare your RMSE to sklearn's GradientBoostingRegressor with the same settings.